SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier[1 [0000−0002−6475−806X]] and Ruben Taelman[1 [0000−0001−5118−256X]]

Department of Electronics and Information Systems, Ghent University – imec

Abstract. Optimizing queries in decentralized environments, where data sources are often unknown and dynamically discovered, remains a major challenge due to the lack of prior knowledge about data distribution and availability. To build user-facing applications on top of such decentralized data, client-side query engines offer a natural place to handle data integration. Because these user-facing applications repeatedly use the same client-side query engine, they create unique optimization opportunities. User-specific engines can capitalize on these opportunities by exploiting patterns in previously issued queries. However, existing benchmarks fail to accurately capture the specific characteristics and behavioral variability of user-centric workloads, relying instead on repetitive or template-based query structures. Moreover, publicly available query logs from real decentralized applications are scarce or absent in the literature. Consequently, the evaluation of client-side decentralized query optimization techniques remains incomplete and cannot be performed under representative conditions. To fill this gap, we introduce an evidence-based simulation of client query usage patterns and offer it as a reusable benchmark. This benchmark enables systematic evaluation and comparison of client-side optimization techniques under realistic and reproducible conditions. We demonstrate its use by applying it to a novel caching-based link traversal query processing approach. To assess the impact of realistic query sequences, we compare the results against a benchmark that is not sequence-based, highlighting how query sequence simulation influences client-side optimization performance. We find that our benchmark effectively reveals critical performance dynamics, such as the impact of session switching and query refinement, which traditional workloads fail to capture. We conclude that modeling realistic user interactions provides an essential foundation for the accurate evaluation and development of client-side optimization techniques in decentralized environments.

Keywords: Link Traversal-based Query Processing · Benchmarking · Workload Generation · Caching.

1 Introduction

Emerging web applications increasingly rely on decentralized architectures to give users control over their personal data, requiring query engines to dynam-

ically fetch and integrate information distributed across the web. While centralizing this data simplifies query processing, it often conflicts with privacy requirements and fine-grained access controls. Structured decentralized ecosystems such as Solid [46] and Mastodon [43] encourage users to keep their own data in small, independent stores that follow shared data models. This approach strengthens user autonomy, yet it makes query execution more difficult because engines must identify and query many heterogeneous data sources, each with its own access policies.

Traditional federated querying approaches [49,47,30] support decentralization of sources, but typically assume a small (10-100) set of well-described and consistently available endpoints [16]. These assumptions fail once data is spread across thousands of small units that enforce local access-control policies. In addition, enforcing fine-grained access control policies [20] on large sources with expressive interfaces introduces notable overhead [40], making standard approaches unsuitable for highly decentralized environments.

Newer decentralized environments, such as Solid, often expose data as lightweight HTTP resources instead of highly expressive query interfaces. Many of these resources are not known in advance and can only be discovered during query execution. Their simplicity lowers the cost of enforcing fine-grained controls. This setting requires a federation model that can operate over a very large number of small sources, many of which may be encountered at runtime, without relying on prior aggregation. Traditional federated approaches are insufficient in this setting due to the assumption of a small number of large endpoints.

Executing SPARQL queries using Link Traversal-based Query Processing (LTQP) [28] fits these needs. It discovers documents incrementally by following URIs encountered during execution, starting from an initial set of seed documents provided by either the user or the query. Its link-driven, asynchronous traversal supports fine-grained permissions and avoids the requirement to know the entire data distribution beforehand.

However, the fact that LTQP discovers documents incrementally creates significant challenges for efficient execution. Using the traditional optimize-then-execute paradigm is difficult, as the query itself determines which data will be accessed. As a result, the optimizer has no prior knowledge of the relevant data until the query's traversal has already begun.

Some approaches optimize query execution without prior knowledge by using adaptive query processing [25] and zero-knowledge query planning [27]. These yield modest but inconsistent performance gains, depending on the decentralized environment and the heuristics used [53]. However, other aspects of zero-knowledge LTQP optimization, such as link prioritization [29], are unlikely to improve query result arrival times in emerging decentralized environments [19].

Consequently, other works explore using precomputed server-side information as prior knowledge or relevance indexes to efficiently execute LTQP queries [54,55,24]. While these approaches offer greater performance benefits, they require the server to maintain these indexes. This raises the cost of serving resources and complicates privacy and access control.

To avoid server-side overhead, engines can derive prior knowledge directly on the client. In decentralized settings, LTQP is inherently client-side, with a single engine instance serving one or more applications. As these applications issue sequences of correlated queries, the engine identifies frequently accessed data and its characteristics. This allows engines to implement *personalized* query optimization algorithms that leverage local access patterns to build necessary prior knowledge. Benchmarking these approaches requires realistic query sequences. Traditional benchmarks (e.g., WatDiv [1], BSBM [10]) execute isolated or repeated query templates, which fails to capture client-side dynamics accurately. Furthermore, while centralized SPARQL optimization relies on real query logs [9,42,11,50], such logs do not yet exist for decentralized LTQP environments. Consequently, we need a standardized benchmark that realistically simulates user query sequences to prevent fragmented evaluation practices.

In this paper, we address this gap in existing benchmarks by introducing SolidSeqBench, a new benchmark that builds upon SolidBench [53] to generate realistic query sequences. These sequences capture three usage patterns identified in social network application literature, the domain simulated by SolidBench, alongside patterns observed in real-world SPARQL logs. Consequently, our benchmark enables researchers to evaluate personalized query optimization techniques against the actual user behaviors they will encounter in real client-facing applications.

To demonstrate our benchmark's utility, we implement three baseline client-side caching strategies in the LTQP engine Comunica [51,53]. We use these intentionally simple algorithms to establish a performance baseline and highlight the necessity of sequence-based evaluation.

2 Literature Review

2.1 Existing Benchmarking Approaches

For the evaluation of client-side benchmarking techniques that rely on preceding queries, we primarily investigate caching literature, as this is a well-studied problem relevant to our work.

In the literature, we find three distinct benchmarking approaches: simulated micro benchmarks, simulated end-to-end query benchmarks, and real-world query logs.

Simulated micro benchmarks isolate specific execution variables, such as cache size [26] or indirectly, cache hit rate [38]. While useful for analyzing isolated mechanisms, they provide an incomplete picture of performance under realistic, dynamic workloads.

Simulated end-to-end benchmarks evaluate overall behavior using query sequences. However, these sequences are typically constructed by randomly grouping isolated queries [39,10,26,38,22], applying simple statistical distributions like Pareto [57,34,6], or relying on narrow application scenarios [14,34] with custom build query sequences. Each of these methods suffers from distinct limitations.

Random grouping ignores real-world behavior entirely. Simple statistical distributions are too basic to capture complex user habits. Finally, static scenarios generate too few queries to provide comprehensive benchmark coverage. Consequently, none of these approaches simulate how users actually explore data, missing how they group related queries into short sessions and constantly adjust their search parameters based on immediate results.

Real-world query logs provide the gold standard for evaluation. While extensive logs are widely used to benchmark centralized SPARQL endpoints [45,50,9,58,41,32], none currently exist for decentralized LTQP environments.

To avoid the fragmented evaluation practices of current simulated approaches, the next best option is a unified simulated benchmark. A shared benchmark would provide a common basis for comparison, improve reproducibility, and prevent each paper from having to create its own evaluation methodology.

2.2 Real-world Query Usage Patterns

To inform the benchmark design on what patterns should be simulated, we will outline three relevant client query usage patterns observed in the literature. The relevance is determined for our social network application use case.

User Interest-Based Query Behavior Work on social networks shows that users tend to form communities based on shared interests [36,15,7,31]. These studies also show that user–user and user–content graphs exhibit power-law degree distributions, small-world structure, and scale-free properties. The result is a dense core of high-degree nodes that connect many smaller, strongly clustered groups of low-degree nodes [36]. Simulation studies of social behavior further support the idea that interest-driven group formation is stable and can be modeled reliably [2,23].

Research shows that users interact more frequently with others who share similar interests [37]. As noted in the study:

> This confirms in a more formal way that shared interests between users are reflected in a higher degree of interaction between them—in an implicit or explicit manner.

These findings support the simulation of user-relatedness and interest-driven behavior when generating query sequences.

Logical Query Sessions Analysis of web browsing behavior shows that activity can be grouped into sessions [35,44,48]. Rather than being defined purely by time, these sessions are better represented as logical sessions [35,48], which capture activity focused on a specific topic or goal.

Rather than being defined purely by time, web browsing activity is better represented as *logical sessions* that capture sequences of user interactions focused on a specific topic or goal [35,48]. These task-oriented sessions frequently involve non-linear navigation patterns, including backtracking. New sessions start in approximately 15% of clicks and follow a log-normal distribution. The average size and depth of logical sessions have log-normal means of 6.1 and 3, respectively.

These same patterns are also exhibited by queries generated during application navigation. Consider the use-case of a social media application built upon decentralized data stores. When users view their feed, they might interact by clicking the username of a post's creator, viewing the comments, or liking the post. In each instance, the parameters of the newly generated query are derived directly from the results of the preceding 'feed' query. These queries are thus chained together in a structure that mirrors logical web sessions. Furthermore, users might search for specific posts, people, or topics, reflecting standard logical session-switching behavior.

Consequently, interactive application usage generates sequences of interdependent requests [56], where subsequent queries rely heavily on preceding results. This is reflected in Facebook's TAO data store, which is explicitly optimized for workloads where data access patterns are driven by the user's step-by-step traversal of the social graph within the application interface [13].

Query Refinement Analysis of SPARQL queries issued to the DBpedia endpoint reveals that users frequently submit related queries in streaks [12,59]. These streaks consist of query sequences that share an underlying intention but undergo iterative refinement to achieve the user's objective. Previous studies initially introduced the concept of SPARQL query sessions specifically to analyze robotic queries [59,12].

In this benchmark, we focus on organic query sequences [59], which users generate through interactions with software or browsers [33]. Because organic interactions are driven by application design rather than underlying data organization, we assume these user behaviors will also emerge in applications navigating structured decentralized environments Furthermore, we deprioritize robotic queries because their sequences can already be simulated by traditional benchmark approaches [53,10,1].

For both organic and robotic queries, analysis of total usage and reformulations (removals and additions) of operators over all query logs shows that filter, union, and optional operators account for the vast majority of reformulated queries.

Recent analyses of Wikidata query logs demonstrate that query development is an iterative process of design, debugging, and maintenance [5]. Based on this empirical data, exploratory querying encompasses six core behaviors [5]: **result refinement** to reduce records via filters or selective triple patterns, **result generalization** to retrieve more entities by broadening parameters, such as adding superclasses or increasing limits, **result extension** to fetch additional attributes by appending new triple patterns, **bug fixing** to correct structural errors like invalid URIs, **query refinement** to adjust complex language features, and **shifting query intent** to pivot topics by altering core predicates.

These findings confirm that real-world querying relies on dynamic, step-by-step modifications rather than the static execution of independent templates. Consequently, existing evaluation frameworks fail to capture how users actively interact with data systems. Evaluating system performance for these exploratory query sequences remains a critical research challenge, explicitly requiring new

benchmarks to validate progress [5]. Therefore, modeling these empirical sequence patterns is essential to accurately assess client-side query optimization techniques.

3 Query Sequence Benchmark

In this section, we detail our simulation methodology and the resulting hyperparameters. We base our sequence generator on SolidBench [53], which simulates realistic decentralized social media applications within the Solid ecosystem. SolidBench utilizes the Social Network Benchmark (SNB) dataset [18,3] and applies the Linked Data Benchmark Council (LDBC) choke point methodology [4]. Its workload combines standard SNB *interactive* queries with custom *discover* templates, which explicitly target Link Traversal Query Processing (LTQP) choke points in decentralized environments.

To support sequence-based evaluation, we extend three core components of the SolidBench ecosystem:

- **Query Generator:** We modify the generator to produce correlated query sequences that reflect empirical real-world usage patterns.
- **Dataset Generator:** We enhance the data generation process to model interest-based similarities between simulated users.
- **Benchmark Runner:** We integrate sequence execution logic into the existing runner to facilitate straightforward, reproducible sequence-based experiments and allow easy generation of default configuration files.

3.1 Simulation Methodology

Our methodology models three interdependent behaviors: task-oriented logical sessions, user interests for data locality, and refinement patterns for iterative query development.

Logical Query Sessions To model logical sessions, we categorize queries into four primary topics: person-specific messages, forum information, person attributes, and message attributes. Because queries in decentralized environments often depend on preceding results, we map the output types of each template to valid subsequent templates. This creates a directed graph of query progression that governs the flow of a logical session.**RT**: If you can't add a figure in the introduction, I'd add one here, to illustrate what you explain in this paragraph.

Within a session, a template's *instantiation values* are typically derived from the previous query's results. To realistically simulate this "click-through" behavior, we execute centralized equivalents of the LDBC SNB queries using QLever [8]. We then randomly sample the returned result set to instantiate the subsequent query template. If a query yields no results, we fall back to our default, interest-based instantiation methodology.

Finally, we simulate logical session switching. Drawing from web browsing behavior, we assign each sequence a session-switching probability sampled from a log-normal distribution with a mean of 15% [35,48]. During sequence generation, a session switch triggers a random choice with equal probability: the engine either starts a new session (using interest-based instantiation) or resumes a previous one (deriving instantiation values from that session's last executed query). To prevent repetitive queries and maintain a uniform distribution of template instantiations, we explicitly prohibit backtracking within our model. Additionally, we maintain a balanced template distribution by dynamically scaling down each template's selection probability proportionally to its overall frequency.

User Interest Modeling Individuals interact more frequently with users sharing similar interests [37]. To simulate this behavior, we use the underlying LDBC dataset utilized by SolidBench [18], which inherently models realistic user preferences. The LDBC generator, Datagen, produces person profiles containing specific interests, educational backgrounds, and workplaces. It links users based on these demographic and interest overlaps, yielding a non-random network topology with a realistic degree distribution.

We use this interest-driven topology to generate template instantiation values for the initial query of each logical session. (As established, subsequent queries derive their values from preceding result sets). Specifically, we construct a binary feature vector v_k for each user k:

$$v_k = (x_{k,1}, \ldots, x_{k,n}, y_{k,1}, \ldots, y_{k,m}), \tag{1}$$

where $x_{k,j} = 1$ if user k holds stated interest j, and $y_{k,l} = 1$ if user k completed study l.

For each sequence, we randomly select a base person k and compute the cosine similarity between v_k and the feature vectors of all other users in the dataset.

We then select template instantiation values probabilistically based on these similarity scores by applying a softmax function with a temperature parameter T:

$$P(c_{i,k}) = \frac{e^{s_i/T}}{\sum_{j=1}^{N} e^{s_j/T}}, \tag{2}$$

where s_i represents the similarity score for candidate person i. To minimize computational overhead at large scale factors, we restrict this softmax operation to the top N candidate entities (ranked by similarity) rather than the entire population.

Finally, for query templates requiring non-person instantiation values (such as posts or activities), we compute selection probabilities based on the similarity scores of the users who created that content.

Refinement Patterns The benchmark simulates query refinement by randomly applying composable transformations to an instantiated query template. Following our literature review, we specifically focus on transformations to add or modify BGPs, OPTIONALs, UNIONs, and query instantiation values.

To ensure these transformations meaningfully alter query execution characteristics, rather than superficially appending triples that maintain a one-to-one relation with the result set, we apply a choke point-based design methodology [4]. Specifically, we define several planning choke points for client-side query optimization in decentralized environments, building on the original SolidBench choke points [53]: **P.1** involves adding or removing a `FILTER`, which requires prior knowledge to adjust query plans and estimate selectivity; **P.2** addresses adding or removing `UNION` or `OPTIONAL` clauses to alter filter push-down capabilities; **P.3** covers adding or removing selective triple patterns; and **P.4** focuses on adding or removing non-selective triple patterns.

Beyond planning, these refinements also impact traversal processes. We identify the following traversal choke points: **T.1** expands or contracts the reachable sub-web through predicate additions; **T.2** shifts the sub-web by substituting the traversal starting point; **T.3** changes the number of hops required to obtain relevant data, corresponding to SolidBench choke points L.1–L.6 [52]; and **T.4** alters the usability of structural indexes, such as type indexes, corresponding to SolidBench choke point L.8 [52].

The generator applies these refinements sequentially to a base SolidBench query template. By default, every refinement pattern includes a reciprocal "undo" operation. The supported refinement operators include:

- **OPTIONAL and UNION blocks:** Adding new blocks or inserting triples into existing ones (including specific `UNION` branches).
- **Basic Graph Patterns (BGPs):** Adding new triple patterns to existing BGPs; the sets of triple patterns that form the core graph-matching logic of a SPARQL query.
- **FILTER statements:** Appending new query constraints.
- **Value substitution:** Replacing template instantiation values to start traversal from a different seed document.

The generator also supports variable instantiation within refinements, allowing users to bind pattern variables to specific values randomly selected from candidates provided through configuration files. Users can define custom query refinement patterns using JSON files to augment the provided default configurations. These defaults, along with documentation and pattern descriptions, are available on GitHub.[1]

Through this flexible generator, we explicitly simulate the empirical exploratory usage patterns [5] identified in Section 2.2. We map these empirical patterns directly to our composable query transformations and choke points:

[1] https://github.com/RubenEschauzier/SolidBench.js/tree/feature/user-based-query-sequences/templates/refinements

- **Result Refinement:** We simulate this by adding or removing `FILTER` constraints and selective triple patterns in the BGP to adjust query selectivity (Choke points P.1, P.3).
- **Result Generalization:** We replicate this by adding new triples to existing BGPs (Choke points P.3, P.4), mirroring how users expand queried classes and fetch related entities.
- **Result Extension:** We simulate this by adding triple patterns to BGPs, `OPTIONAL` blocks, and `UNION` blocks (Choke points P.3, P.4, P.2).
- **Query Refinement:** We model this through the addition or removal of `OPTIONAL` and `UNION` blocks (Choke point P.2).

As our refinement rules are fully composable, the generator naturally reflects the empirical observation that exploratory patterns are not mutually exclusive and frequently co-occur within a single session.

3.2 Hyperparameters

The sequence generation process introduces several new hyperparameters, which are detailed in Table 1 together with their default values. To guarantee reproducibility despite extensive random sampling, a single seeded pseudo-random number generator (PRNG) is used througout the entire pipeline.

Table 1. LogT-normal distribution parameters (μ, σ) control the queries per sequence, queries per logical session, and the probability of switching logical sessions. Refinement parameters define the likelihood (`patternProbability`) and bounds (`min/maxRefinementLength`) of generating a refinement sequence for a given query. Instantiation relies on a softmax `temperature` and a `maxLogits` cutoff for similarity-based entity selection.

Category	Parameter	Value
Distributions (μ, σ)	Sequence Length	3.0, 0.2
	Session Length	1.7, 0.5
	Transition Probability	-2.0, 0.25
Refinements	`patternProbability`	0.1
	`min/maxRefinementLength`	2, [3–5]
Instantiation	`temperature`	0.1
	`maxLogits`	5–10

4 Caching in Link Traversal-based Query Processing

To validate the query sequences generated by the SolidBenchSequence (SBS) benchmark, we implemented three caching approaches in the Comunica link

traversal engine [51,53]. Specifically, we cache the quads from documents dereferenced during link traversal. High cache hit rates eliminate redundant HTTP requests, increasing data availability and reducing query latency. To evaluate how different cache management strategies impact this latency, we compare the following three configurations:

Unindexed Cache This baseline approach caches the raw quads obtained during traversal. Upon a cache hit, these quads are retrieved and indexed into the active local store for query execution.

Indexed Cache Alternatively, we index quads before caching them. Upon a cache hit, the engine extracts only quads matching the quad patterns in the current query. While this drastically reduces the memory footprint and indexing overhead of the local store, it increases the initial cache ingestion cost. We hypothesize this trade-off is only beneficial given sufficiently high hit rates.

Indexed Cache-based Estimation Because the cache is indexed, it can execute count queries to estimate quad pattern cardinalities. Traditional LTQP relies on heuristics due to a lack of prior knowledge [27]. By estimating cardinalities against *all* cached documents, regardless of their reachability for the current query, we can replace these heuristics with Comunica's default cost-based optimizer [51] to generate more accurate query plans.

For each caching algorithm, we investigate three sizes: small (350000 quads), medium (700000 quads), and large (1400000 quads). These triple counts correspond to about 10%, 20%, and 40% of the total generated dataset size.

Our implementations adhere to the RFC-9111 specification [21]. However, because the benchmark dataset is static, cache entries require no revalidation and never become stale. Consequently, the observed hit rates are likely higher than in a live environment. Furthermore, while the fundamental concept of caching in link traversal is established [26], integrating and comparing these distinct caching strategies provides novel and strong baselines for evaluating caching in traversal engines, upon which future research into advanced caching approach can build.

5 Evaluation

We evaluate our benchmark using a SolidBench dataset at a scale of 0.1, generating 158,233 RDF files across 1,531 data vaults with a total of 3,556,159 triples. We generate 8 query sequences totaling 192 query instances using the default hyperparameters in Table 1. We run all experiments on a 64-bit Ubuntu 20.04.6 machine with an Intel Core i5-9400 CPU at 2.90GHz, allocating a maximum of 16 GB of RAM to the query engine and enforcing a 180-second timeout per query. We repeat each sequence 10 times to reduce measurement variance. Because the server and client share the same machine, we simulate realistic network conditions by introducing a uniformly sampled latency between 70 and 90 ms.

Our experiments serve a primary and a secondary purpose. Primarily, we evaluate the simulated query sequences to determine how our three design choices impact query correlation. Because caching algorithms primarily express this correlation in *hit rates* and *evictions*, we validate the benchmark by these metrics. Secondarily, we apply the implemented caching approaches to establish a performance baseline for caching in a single-threaded LTQP engine. While we report the observed execution speedups to demonstrate the benchmark's utility, our focus remains on validating the sequence generation rather than conducting an exhaustive performance analysis of the caching algorithms themselves.

5.1 Global Cache Performance

Figure 1 presents the global caching performance profiles [17] for each approach at various sizes against the no-cache baseline. As expected, all caching methods outperform the baseline.

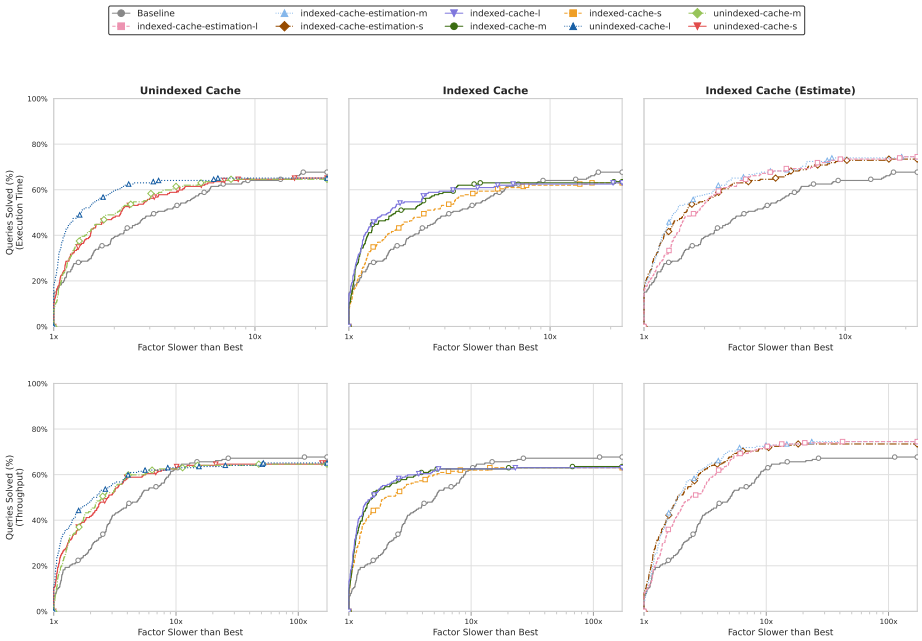

Fig. 1. Performance profile of the tested cache algorithms. The y-axis shows the percentage of queries executing within a given performance factor (x-axis) of the fastest algorithm for that specific query. To interpret the plot: curves closer to the top-left corner indicate better overall performance. The y-intercept ($x = 1$) represents the percentage of queries where an algorithm is the outright fastest. Conversely, the rightward trajectory demonstrates robustness. For example, a point at $y = 80\%$ and $x = 2$ indicates the algorithm executes 80% of the queries at most twice as slowly as the best available method.

For the unindexed cache, small (350,000 triples) and medium (700,000 triples) sizes perform similarly. The large cache (1,400,000 triples) shows significant improvement, indicating it reaches the critical capacity needed to retain entries across the entire query sequence. However, the baseline *solves* more total queries. This indicates that cache management overhead causes some long-running queries to fail before the deadline.

The indexed cache results support this, showing an even higher proportion of failed queries. This aligns with the hypothesis that higher indexing costs slow down execution during frequent cache evictions. Yet, when the indexed cache succeeds, it provides a larger performance boost than the unindexed cache. Additionally, the indexed cache has a lower critical capacity threshold, as medium and large sizes perform similarly.

Finally, the indexed cache with cardinality estimation performs best. It significantly improves query speeds and solves *more* queries than the baseline. For this method, increasing the cache size does not improve performance; in fact, the large cache performs worse than the small and medium versions. This suggests cardinality estimation benefits from a smaller, less stale cache, because the estimation process evaluates all cache entries rather than just traversal-relevant ones.

5.2 Logical Sessions

To evaluate the impact of logical sessions on caching performance, we analyze the hit rate deviation (Δ) from the overall average when queries initiate a new session, switch to an existing one, or remain in the current session. Table 2 shows that new sessions severely degrade hit rates, existing session switches cause moderate degradation, and intra-session queries improve performance. These effects are strongest in small and medium caches.

Consequently, optimal client-side cache sizing depends heavily on user session-switching behavior. These findings highlight the need for eviction policies that better retain cross-session entries and validate our simulation design; ignoring session dynamics would obscure critical real-world performance shifts.

Future work should investigate whether query sequences share a frequently accessed core of cache entries. Protecting these entries from eviction could mitigate the performance penalties of session switching.

5.3 Refinement Patterns

To evaluate iterative query refinement simulation, we analyze cache behavior across refinement depths. Figure 1 shows that hit ratios increase progressively as queries undergo sequential refinement. Across all algorithms, hit rates increase at depths 2 and 3, confirming that refinement heavily reuses prior data.

At higher refinement depths, performance depends heavily on cache capacity. Large caches sustain high hit rates through depth 4, whereas small caches experience significant thrashing, and degraded hit ratios.

Table 2. Impact of logical session status on algorithm performance. Values represent absolute and percentage cache hit rate deviations from the global template average across new, existing, and within-session queries.

Cache Algorithm	New Session		Existing Session		Within Session	
	Abs.	%	Abs.	%	Abs.	%
unindexed-l	-0.066	-12.278	-0.014	-2.474	0.011	1.931
unindexed-m	-0.056	-15.649	-0.031	-7.900	0.024	5.486
unindexed-s	-0.071	-24.275	-0.021	-6.551	0.017	4.582
indexed-estimate-l	-0.053	-10.573	-0.015	-2.796	0.012	2.127
indexed-estimate-m	0.028	7.174	-0.000	-0.064	-0.000	-0.035
indexed-estimate-s	-0.044	-14.305	-0.013	-3.921	0.010	2.924
indexed-l	-0.026	-5.011	-0.003	-0.571	0.003	0.467
indexed-m	-0.087	-22.782	-0.021	-5.098	0.017	3.682
indexed-s	-0.033	-11.036	-0.015	-4.470	0.012	3.153

Surprisingly, the indexed cache without estimation maintains high hit rates even with smaller capacities. This counterintuitive result stems from implementation differences that influence how the single-threaded Node.js event loop schedules asynchronous traversal and join executions. This scheduling variance alters traversal order and rate, ultimately impacting cache eviction and hit performance.

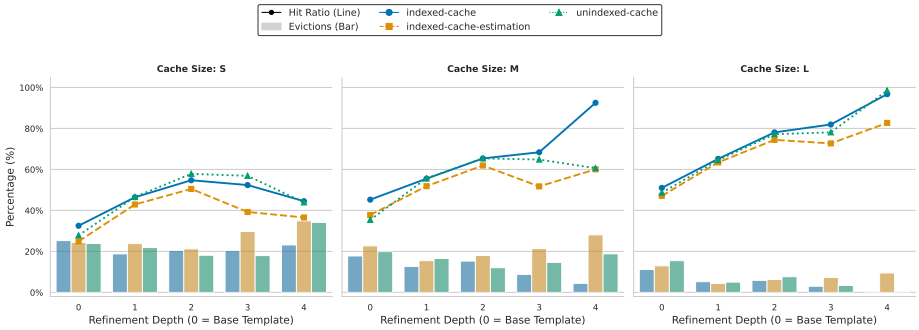

Fig. 2. Cache hit ratios and percentage of cache capacity evicted across refinement depths for small, medium, and large cache configurations.

The observed cache hitrate and eviction differences directly translate to measurable execution speedups. Figure 2 illustrates that the geometric mean speedup for mainly throughput generally scales positively with refinement depth. The indexed-cache algorithm proves particularly effective for prolonged exploratory

sequences, achieving substantial throughput gains at depth 4 compared to the unindexed alternatives.

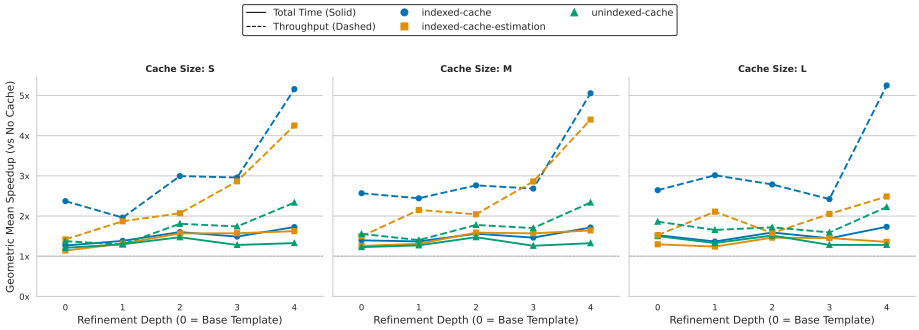

Fig. 3. Geometric mean of observed speedups and increases respectively for total execution time and query throughput relative to the no caching baseline across refinement depths.

5.4 Theoretical Hit Rate Analysis

To analyze theoretical caching speedup without management costs, we executed one instance of each query template twice and simulated cache misses at varying probabilities. We excluded the estimation-based approach because using the full cache for estimation would skew the results.

From Figures 4 and 5, we find that in most cases caching can significantly improve query execution time, with improvements scaling predictably as hit rate increases. For some long-running queries such as interactive-discover-6, 7, and short-3 no speedup is observed, with some queries even showing slower result production. These query templates traverse many solid pods, but do not require all traversed data. In such cases, we hypothesize that having access to a cache which serves documents from memory will quickly produce too much data and slow down result production. Thus, waiting for HTTP responses between traversal steps allows the JavaScript event loop to run the join execution pipeline and thus produces results faster as only the first few requests are required for first results.

The two caching approaches exhibit similar overall performance, though the indexed cache significantly reduces the time-to-first-result across most templates. As expected, an indexed cache is highly advantageous when in the absence of evictions. However, the benchmark results demonstrate that the initial cost of indexing data introduces a clear trade-off: it amplifies both the potential speedups and slowdowns compared to an unindexed cache.

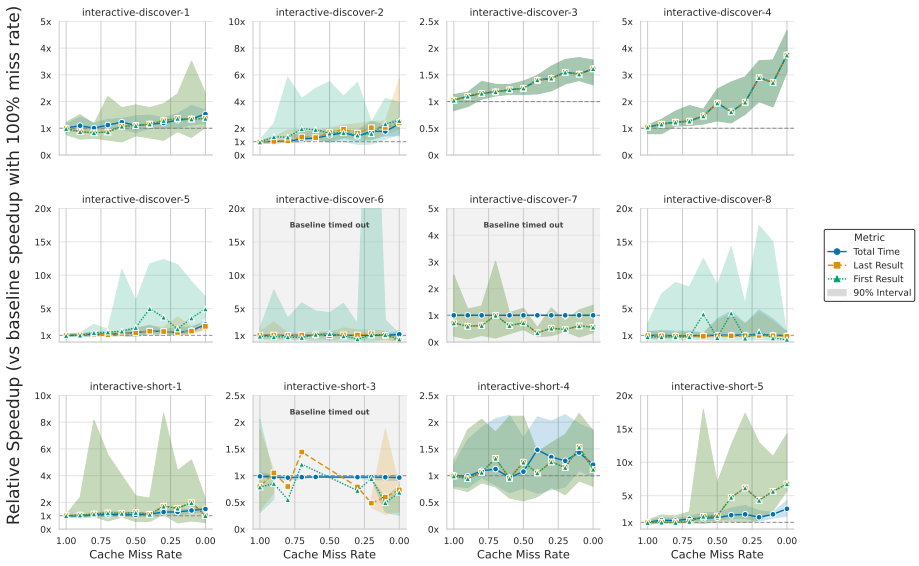

Fig. 4. Theoretical unindexed cache performance by query template. Lines indicate the mean relative speedup of the repeated query for total execution time, first result, and last result against a 100% miss rate baseline. Shaded regions denote the absolute minimum and maximum variance across iterations.

5.5 Benchmark Abalations

We perform ablation studies to investigate design choices not evaluated in the preceding subsections. Figure 6 shows execution times for sequences lacking our simulation methodology, created by randomly grouping default SolidBench.js queries. For these random sequences, only large caches perform well, while smaller ones degrade below the baseline. These results misleadingly suggest that large caches are vital, whereas our benchmark demonstrates significant performance benefits even with smaller sizes. As expected, the estimation cache performs well in both scenarios because cardinality estimation does not require cache hits.

6 Conclusion

{**TODO**: Mention maintenance plan about how we have PRs open to add to SolidBench and how SolidBench has been maintained for several years now, so by tying this resource's fate to that we have solid track record}

References

1. Aluç, G., Hartig, O., Özsu, M.T., Daudjee, K.: Diversified stress testing of rdf data management systems. In: International Semantic Web Conference. pp. 197–212. Springer (2014)

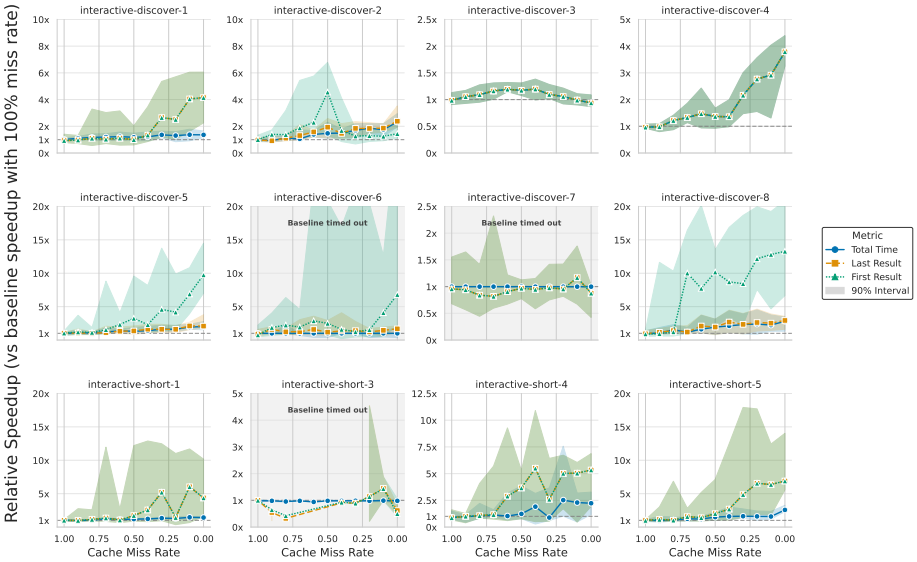

Fig. 5. Theoretical indexed cache performance by query template. Lines indicate the mean relative speedup of the repeated query for total execution time, first result, and last result against a 100% miss rate baseline. Shaded regions denote the absolute minimum and maximum variance across iterations.

2. Amblard, F., Bouadjio-Boulic, A., Gutiérrez, C.S., Gaudou, B.: Which models are used in social simulation to generate social networks? a review of 17 years of publications in jasss. In: 2015 Winter Simulation Conference (WSC). pp. 4021–4032. IEEE (2015)
3. Angles, R., Antal, J.B., Averbuch, A., Birler, A., Boncz, P., Búr, M., Erling, O., Gubichev, A., Haprian, V., Kaufmann, M., et al.: The ldbc social network benchmark (2020)
4. Angles, R., Boncz, P., Larriba-Pey, J., Fundulaki, I., Neumann, T., Erling, O., Neubauer, P., Martinez-Bazan, N., Kotsev, V., Toma, I.: The linked data benchmark council: a graph and rdf industry benchmarking effort. vol. 43, pp. 27–31. ACM New York, NY, USA (2014)
5. Arenas, M., Franconi, E., Hammerer, J., Hartig, O., Hose, K., Koesten, L., Konstantinidis, G., Libkin, L., Martens, W., Sasaki, Y., et al.: Exploring exploratory querying. Proceedings of the VLDB Endowment **18**(13), 5731–5739 (2025)
6. Arnold, B.C.: Pareto distribution. Wiley StatsRef: Statistics Reference Online pp. 1–10 (2014)
7. Backstrom, L., Huttenlocher, D., Kleinberg, J., Lan, X.: Group formation in large social networks: membership, growth, and evolution. In: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining. pp. 44–54 (2006)
8. Bast, H., Buchhold, B.: Qlever: A query engine for efficient sparql+ text search. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. pp. 647–656 (2017)

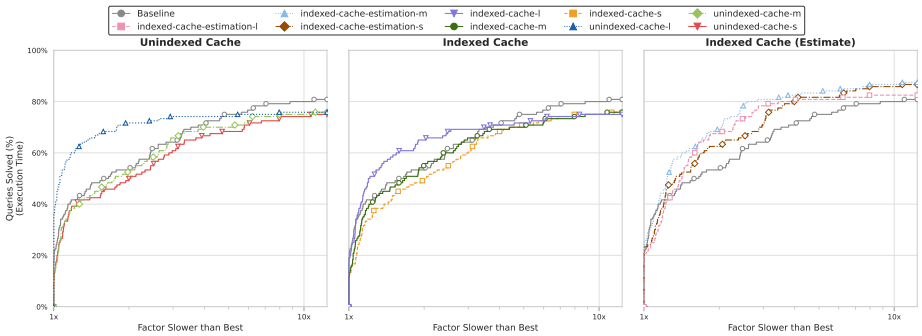

Fig. 6. Performance profile of the tested cache algorithms on random query sequences. The y-axis shows the percentage of queries that execute within a given performance factor (x-axis) of the fastest algorithm for that query.

9. Berendt, B., Hollink, L., Luczak-Rösch, M., Möller, K., Vallet, D.: Usewod2013 3rd international workshop on usage analysis and the web of data. 10th ESWC-Semantics and Big Data, Montpellier, France (2013)
10. Bizer, C., Schultz, A.: The berlin sparql benchmark. In: Semantic Services, Interoperability and Web Applications: Emerging Concepts, pp. 81–103. IGI Global Scientific Publishing (2011)
11. Bonifati, A., Martens, W., Timm, T.: DARQL: Deep Analysis of SPARQL Queries. In: Companion Proceedings of the The Web Conference 2018. pp. 187–190. WWW '18, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE (Apr 2018). https://doi.org/10.1145/3184558.3186975, https://dl.acm.org/doi/10.1145/3184558.3186975
12. Bonifati, A., Martens, W., Timm, T.: An analytical study of large sparql query logs. The VLDB Journal **29**(2), 655–679 (2020)
13. Bronson, N., Amsden, Z., Cabrera, G., Chakka, P., Dimov, P., Ding, H., Ferris, J., Giardullo, A., Kulkarni, S., Li, H., et al.: {TAO}:{Facebook's} distributed data store for the social graph. In: 2013 USENIX annual technical conference (USENIX ATC 13). pp. 49–60 (2013)
14. Chatzopoulos, S., Vergoulis, T., Skoutas, D., Dalamagas, T., Tryfonopoulos, C., Karras, P.: Atrapos: Evaluating metapath query workloads in real time. CoRR (2022)
15. Clark, A.: Understanding community: A review of networks, ties and contacts (2007)
16. Dang, M.H., Aimonier-Davat, J., Molli, P., Hartig, O., Skaf-Molli, H., Le Crom, Y.: Fedshop: A benchmark for testing the scalability of sparql federation engines. In: International Semantic Web Conference. pp. 285–301. Springer (2023)
17. Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. Mathematical programming **91**(2), 201–213 (2002)
18. Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., Pham, M.D., Boncz, P.: The ldbc social network benchmark: Interactive workload. In: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. pp. 619–630 (2015)

19. Eschauzier, R., Taelman, R., Verborgh, R.: Revisiting link prioritization for efficient traversal in structured decentralized environments. In: International Semantic Web Conference. pp. 575–593. Springer (2025)
20. Esteves, B., Pandit, H.J., Rodríguez-Doncel, V.: Odrl profile for expressing consent through granular access control policies in solid. In: 2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). pp. 298–306. IEEE (2021)
21. Fielding, R., Nottingham, M., Reschke, J.: Rfc 9111: Http caching (2022)
22. Folz, P., Skaf-Molli, H., Molli, P.: Cyclades: a decentralized cache for triple pattern fragments. In: European semantic web conference. pp. 455–469. Springer (2016)
23. Hamill, L., Gilbert, G.: Social circles: A simple structure for agent-based social network models. Journal of Artificial Societies and Social Simulation **12**(2) (2009)
24. Hanski, J., Taelman, R., Verborgh, R.: Observations on bloom filters for traversal-based query execution over solid pods. In: European Semantic Web Conference. pp. 228–233. Springer (2024)
25. Hanski, J., Van Braeckel, S., Taelman, R., Verborgh, R.: Link traversal over decentralised environments using restart-based query planning. In: International Conference on Web Engineering (2025)
26. Hartig, O.: How caching improves efficiency and result completeness for querying linked data. In: LDOW (2011)
27. Hartig, O.: Zero-knowledge query planning for an iterator implementation of link traversal based query execution. In: Extended Semantic Web Conference. pp. 154–169. Springer (2011)
28. Hartig, O., Bizer, C., Freytag, J.C.: Executing sparql queries over the web of linked data. In: International semantic web conference. pp. 293–309. Springer (2009)
29. Hartig, O., Özsu, M.T.: Walking without a map: Ranking-based traversal for querying linked data. In: International Semantic Web Conference. pp. 305–324. Springer (2016)
30. Heling, L., Acosta, M.: Federated sparql query processing over heterogeneous linked data fragments. In: Proceedings of the ACM Web Conference 2022. pp. 1047–1057 (2022)
31. Kalaitzakis, A., Papadakis, H., Fragopoulou, P.: Evolution of user activity and community formation in an online social network. In: Proceedings of the 2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining. pp. 1315–1320. IEEE (2012)
32. Lorey, J., Naumann, F.: Caching and prefetching strategies for sparql queries. In: Extended Semantic Web Conference. pp. 46–65. Springer (2013)
33. Malyshev, S., Krötzsch, M., González, L., Gonsior, J., Bielefeldt, A.: Getting the most out of wikidata: Semantic technology usage in wikipedia's knowledge graph. In: The Semantic Web–ISWC 2018: 17th International Semantic Web Conference, Monterey, CA, USA, October 8–12, 2018, Proceedings, Part II 17. pp. 376–394. Springer (2018)
34. Martin, M., Unbehauen, J., Auer, S.: Improving the performance of semantic web applications with sparql query caching. In: Extended Semantic Web Conference. pp. 304–318. Springer (2010)
35. Meiss, M., Duncan, J., Gonçalves, B., Ramasco, J.J., Menczer, F.: What's in a session: tracking individual behavior on the web. In: Proceedings of the 20th ACM conference on Hypertext and hypermedia. pp. 173–182. ACM, Torino Italy (Jun 2009). https://doi.org/10.1145/1557914.1557946, https://dl.acm.org/doi/10.1145/1557914.1557946

36. Mislove, A., Marcon, M., Gummadi, K.P., Druschel, P., Bhattacharjee, B.: Measurement and analysis of online social networks. In: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement. pp. 29–42 (2007)
37. Mitzlaff, F., Atzmueller, M., Benz, D., Hotho, A., Stumme, G.: User-relatedness and community structure in social interaction networks. arXiv preprint arXiv:1309.3888 (2013)
38. Nicholson, H., Chrysogelos, P., Ailamaki, A.: Hpcache: memory-efficient olap through proportional caching revisited. The VLDB Journal **33**(6), 1775–1791 (2024)
39. O'Neil, P., O'Neil, E., Chen, X., Revilak, S.: The star schema benchmark and augmented fact table indexing. In: Technology Conference on Performance Evaluation and Benchmarking. pp. 237–252. Springer (2009)
40. Padia, A., Finin, T., Joshi, A., et al.: Attribute-based fine grained access control for triple stores. In: 3rd Society, Privacy and the Semantic Web-Policy and Technology workshop, 14th International Semantic Web Conference (2015)
41. Papailiou, N., Tsoumakos, D., Karras, P., Koziris, N.: Graph-aware, workload-adaptive sparql query caching. In: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. pp. 1777–1792 (2015)
42. Picalausa, F., Vansummeren, S.: What are real SPARQL queries like? In: Proceedings of the International Workshop on Semantic Web Information Management. pp. 1–6. SWIM '11, Association for Computing Machinery, New York, NY, USA (Jun 2011). https://doi.org/10.1145/1999299.1999306, https://dl.acm.org/doi/10.1145/1999299.1999306
43. Raman, A., Joglekar, S., Cristofaro, E.D., Sastry, N., Tyson, G.: Challenges in the decentralised web: The mastodon case. In: Proceedings of the internet measurement conference. pp. 217–229 (2019)
44. Sadagopan, N., Li, J.: Characterizing typical and atypical user sessions in clickstreams. In: Proceedings of the 17th international conference on World Wide Web. pp. 885–894 (2008)
45. Saleem, M., Ali, M.I., Hogan, A., Mehmood, Q., Ngomo, A.C.N.: Lsq: the linked sparql queries dataset. In: International semantic web conference. pp. 261–269. Springer (2015)
46. Sambra, A.V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., Berners-Lee, T.: Solid: a platform for decentralized social applications based on linked data. MIT CSAIL & Qatar Computing Research Institute, Tech. Rep. **2016** (2016)
47. Schmidt, M., Görlitz, O., Haase, P., Ladwig, G., Schwarte, A., Tran, T.: Fedbench: A benchmark suite for federated semantic data query processing. In: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10. pp. 585–600. Springer (2011)
48. Schneider, F., Feldmann, A., Krishnamurthy, B., Willinger, W.: Understanding online social network usage from a network perspective. In: Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement. pp. 35–48 (2009)
49. Schwarte, A., Haase, P., Hose, K., Schenkel, R., Schmidt, M.: Fedx: Optimization techniques for federated query processing on linked data. In: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10. pp. 601–616. Springer (2011)
50. Stadler, C., Saleem, M., Mehmood, Q., Buil-Aranda, C., Dumontier, M., Hogan, A., Ngonga Ngomo, A.C.: Lsq 2.0: A linked dataset of sparql query logs. Semantic Web **15**(1), 167–189 (2024)

51. Taelman, R., Van Herwegen, J., Vander Sande, M., Verborgh, R.: Comunica: a modular sparql query engine for the web. In: International Semantic Web Conference. pp. 239–255. Springer (2018)
52. Taelman, R., Verborgh, R.: Evaluation of link traversal query execution over decentralized environments with structural assumptions (2023)
53. Taelman, R., Verborgh, R.: Link traversal query processing over decentralized environments with structural assumptions. In: International Semantic Web Conference. pp. 3–22. Springer (2023)
54. Tam, B.E., Taelman, R., Colpaert, P., Verborgh, R.: Opportunities for shape-based optimization of link traversal queries. arXiv preprint arXiv:2407.00998 (2024)
55. Umbrich, J., Hose, K., Karnstedt, M., Harth, A., Polleres, A.: Comparing data summaries for processing live queries over linked data. World Wide Web **14**(5), 495–544 (2011)
56. Vögele, C., van Hoorn, A., Schulz, E., Hasselbring, W., Krcmar, H.: Wessbas: extraction of probabilistic workload specifications for load testing and performance prediction—a model-driven approach for session-based application systems. Software & Systems Modeling **17**(2), 443–477 (2018)
57. Williams, G.T., Weaver, J.: Enabling fine-grained http caching of sparql query results. In: International Semantic Web Conference. pp. 762–777. Springer (2011)
58. Zhang, W.E., Sheng, Q.Z., Qin, Y., Yao, L., Shemshadi, A., Taylor, K.: Secf: improving sparql querying performance with proactive fetching and caching. In: Proceedings of the 31st Annual ACM Symposium on Applied Computing. pp. 362–367 (2016)
59. Zhang, X., Wang, M., Zhao, B., Liu, R., Zhang, J., Yang, H.: Characterizing robotic and organic query in sparql search sessions. In: Web and Big Data: 4th International Joint Conference, APWeb-WAIM 2020, Tianjin, China, September 18-20, 2020, Proceedings, Part I 4. pp. 270–285. Springer (2020)